UNIVERSIDADE
FEDERAL DO CEARÁ

RELATÓRIO: TRABALHO 1

Aluno:
Francisco Werley da Silva - 553948
Camile Isidorio Araújo - 555251

Professor: Rafael Braga
Disciplina: Sistemas Distribuídos

1. Introdução

- O que aprenderam e o que ainda tem dificuldade?
  - Aprendi na prática como funciona a comunicação em sistemas distribuídos utilizando sockets em Java;
  - Entendi como ocorre a troca de dados por fluxo de bytes;
  - Aprendi a implementar serialização manual (empacotamento de dados);
  - Aprendi a fazer desserialização manual (leitura dos dados recebidos);
  - Sem muitas dificuldades.

- Nota que a dupla merece (em uma escala de 0 a 10);
  - 10

- Nível de dificuldade (muito fácil, fácil, média, difícil ou muito difícil);
  - média

- Quantas horas precisaram para completar a atividade? (Total = estudo pessoal + leituras + videoaulas extra + implementação)
  - 6 horas em média

- Caso desejem, deixem alguma sugestão para melhorar a atividade.
  - sem sugestões

- Qual o link do repositório com o código-fonte?
  - LINK

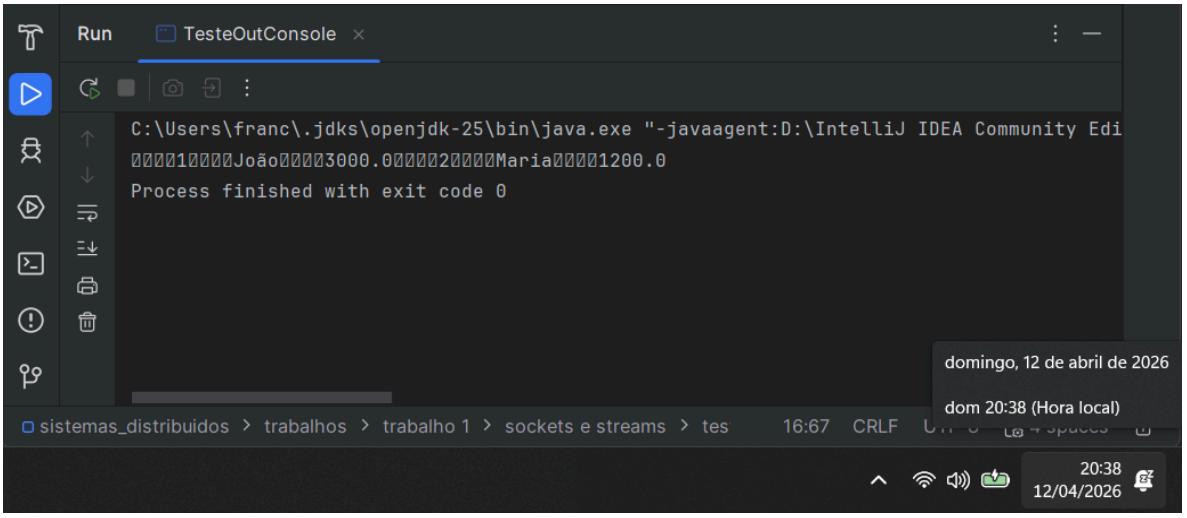
2. Sockets e Streams

Este trabalho apresenta o desenvolvimento de uma aplicação baseada em um Sistema de Gestão de Recursos Humanos utilizando Java, com foco em conceitos de sistemas distribuídos e manipulação de streams. A modelagem do sistema foi estruturada a partir da superclasse Colaborador, contendo atributos básicos como id, nome e salário, e suas subclasses: Funcionário, Estagiário, Autônomo e Efetivo, permitindo especialização e reaproveitamento de código por meio de herança. Além disso, foi criada a interface Admissivel, implementada por algumas subclasses, definindo o comportamento de admissão e possibilitando o uso de polimorfismo. A classe Departamento foi utilizada para representar uma agregação, contendo múltiplos colaboradores, enquanto as classes ColaboradorService e DepartamentoService foram responsáveis por gerenciar os dados e separar a lógica de negócio da modelagem. Como parte central do trabalho, foram implementadas subclasses de OutputStream e InputStream, denominadas ColaboradorOutputStream e ColaboradorInputStream, responsáveis por realizar a serialização e desserialização manual dos objetos. Nesse processo, os dados dos colaboradores (id, nome e salário) foram convertidos para bytes e enviados juntamente com o tamanho de cada atributo, garantindo

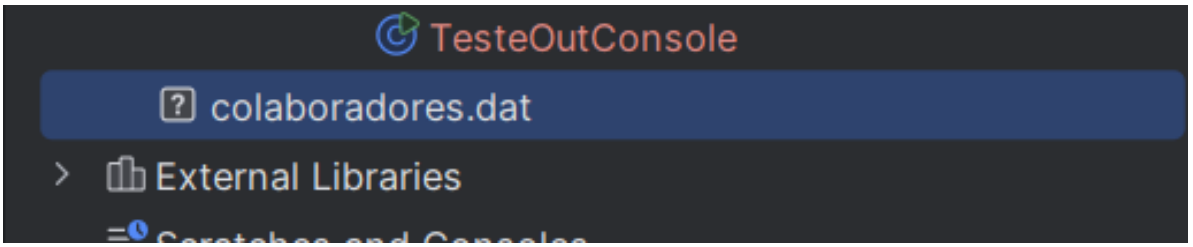
que pudessem ser reconstruídos corretamente na leitura. A aplicação foi testada em diferentes cenários: saída padrão (System.out), leitura via entrada padrão (System.in), manipulação de arquivos (FileOutputStream e FileInputStream) e comunicação em rede utilizando sockets TCP, com implementação de cliente e servidor. Esses testes demonstraram a flexibilidade e a reutilização dos streams em diferentes meios de comunicação. Como resultado, o trabalho proporcionou a compreensão prática de conceitos como serialização manual, comunicação distribuída, uso de streams e organização em camadas, evidenciando também as vantagens e desafios dessa abordagem, como maior controle sobre os dados e aumento da complexidade de implementação.

Prints das saídas dos testes:

TesteOutConsole

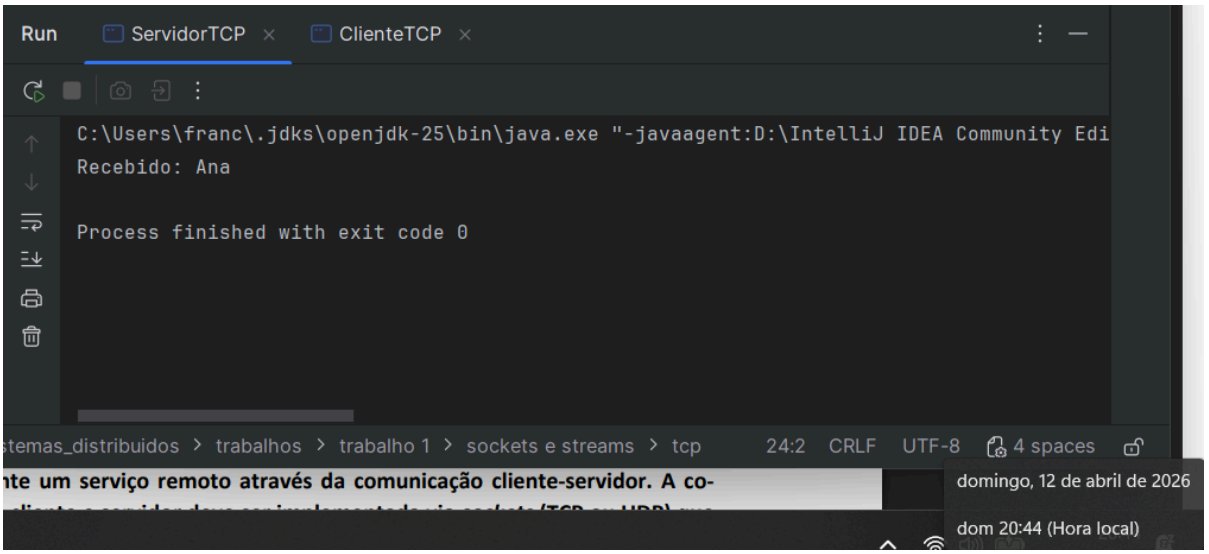


TesteOutArquivo
(o output é o arquivo “colaboradores”)

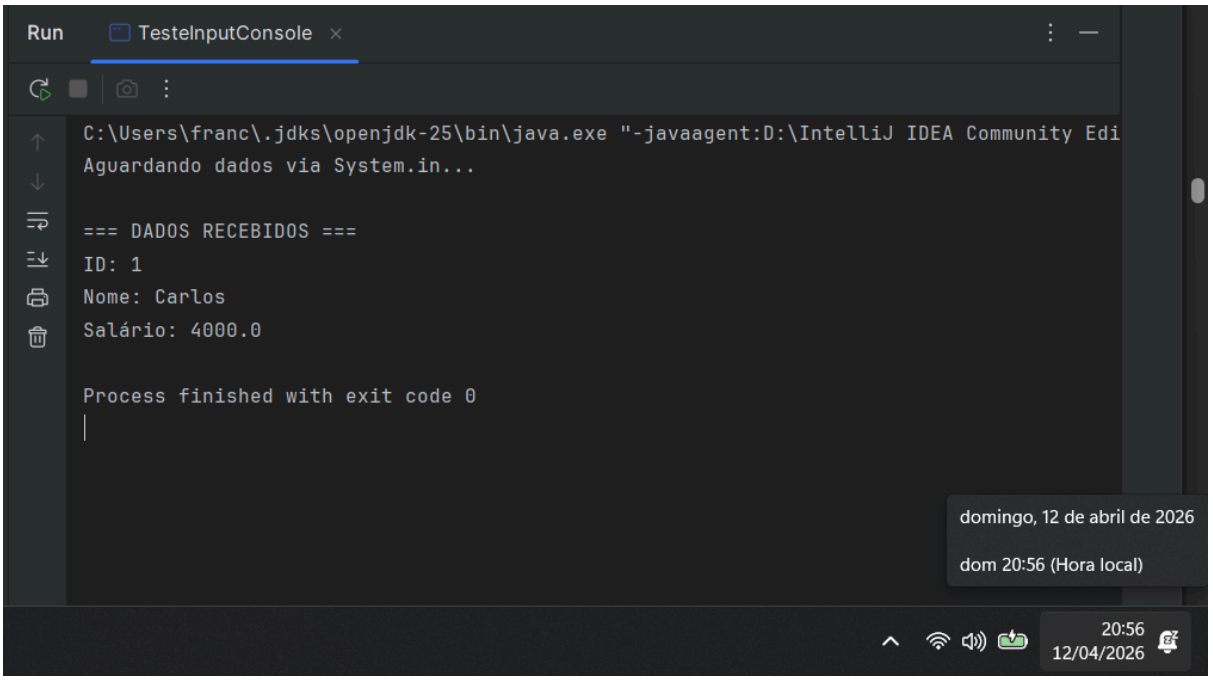


Teste usando conexão TCP

(aqui o teste do input e do output só tem uma saída, porque é preciso que o cliente envie para que tenha saída no servidor)



TesteInputConsole



TesteInputArquivo

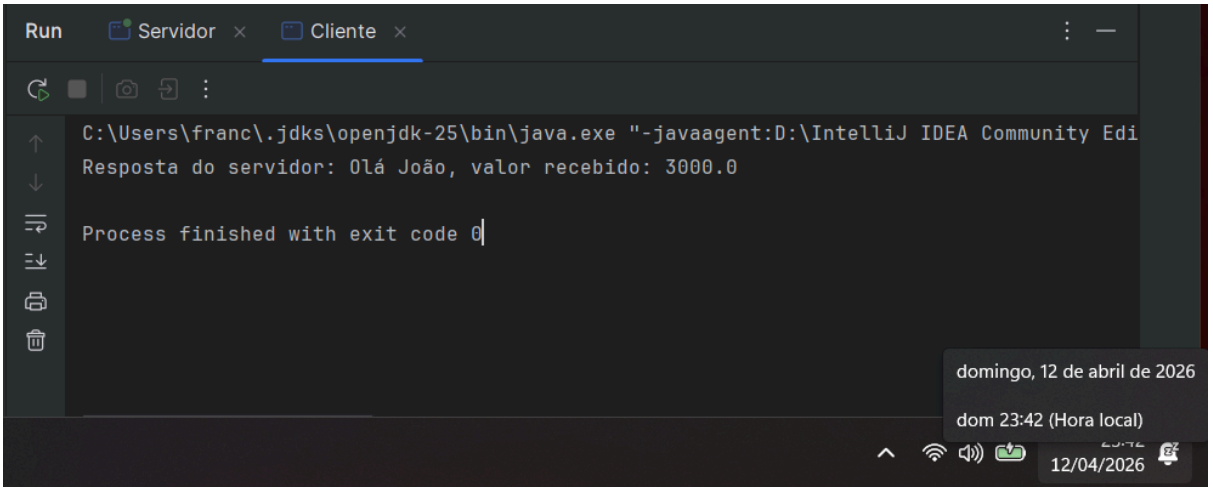
3. Serialização

Este trabalho teve como objetivo implementar um sistema de comunicação cliente-servidor utilizando sockets TCP em Java, com troca de mensagens através de fluxo de bytes e uso de serialização manual. Diferente da serialização automática da linguagem, foi necessário empacotar e desempacotar os dados manualmente utilizando as classes DataOutputStream e DataInputStream. A aplicação foi estruturada em quatro partes principais: o cliente, o servidor e duas classes de dados chamadas Request e Response. O cliente é responsável por criar uma requisição contendo informações como id, nome e valor, serializar esses dados e enviá-los ao servidor. O servidor, por sua vez, recebe essa mensagem, realiza a desserialização, processa as informações e gera uma resposta que é novamente serializada e enviada de volta ao cliente. Ao receber a resposta, o cliente faz a desserialização e exibe o resultado. Durante a implementação, foi importante garantir que a ordem de escrita dos dados fosse a mesma da leitura, pois isso assegura que as informações sejam interpretadas corretamente. Os testes realizados mostraram que a comunicação ocorreu com sucesso, com o cliente enviando dados corretamente e o servidor respondendo de forma adequada. Com isso, foi possível compreender melhor como funciona a comunicação em sistemas distribuídos, especialmente o uso de sockets e a manipulação de dados em nível de bytes. Como melhorias futuras, poderiam ser adicionados suporte a múltiplos clientes utilizando threads, criação de um protocolo de comunicação mais robusto e tratamento de erros mais completo.

Prints das saídas:

Servidor

Cliente



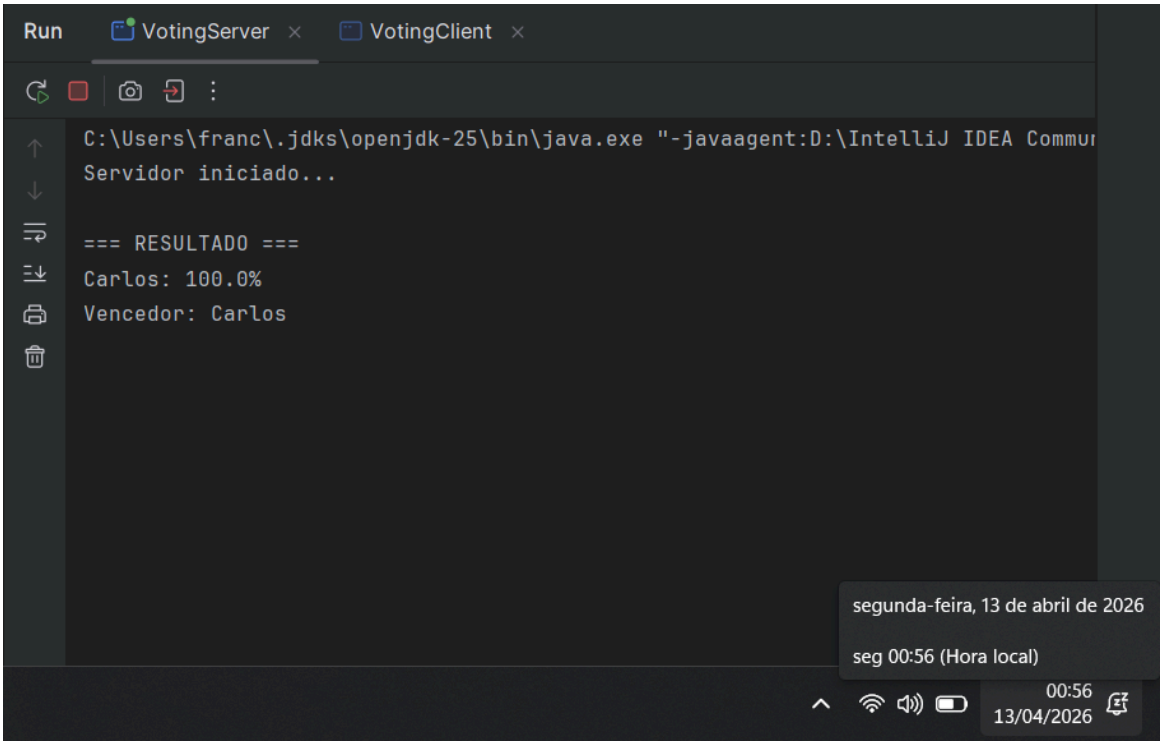
4. Representação externa de dados

A aplicação desenvolvida consiste em um sistema de votação distribuído implementado em Java, utilizando comunicação via sockets TCP e UDP, conforme os requisitos propostos. O servidor foi projetado para ser multi-threaded, permitindo o atendimento simultâneo de múltiplos clientes, onde cada conexão é tratada por uma thread independente. A comunicação TCP foi utilizada para operações que exigem confiabilidade, como o login dos eleitores, envio da lista de candidatos disponíveis e registro dos votos. Ao se conectar, o cliente realiza um login simples, recebe a lista de candidatos cadastrados e, em seguida, pode enviar seu voto, que é processado e armazenado pelo servidor. O sistema possui um controle de tempo para a votação, no qual, após um período definido, o servidor encerra o recebimento de votos automaticamente e realiza o cálculo dos resultados, exibindo o total de votos por candidato, suas respectivas porcentagens e o candidato vencedor. Para o

envio de mensagens informativas aos clientes, foi utilizada comunicação multicast com UDP, permitindo que administradores enviem avisos que são recebidos simultaneamente por todos os clientes conectados ao grupo multicast. A representação dos dados foi feita de forma simples, utilizando troca de mensagens em formato textual, o que facilita a implementação e compreensão do sistema, embora possa ser substituída por formatos mais estruturados como JSON ou Protocol Buffers. A aplicação demonstra conceitos fundamentais de sistemas distribuídos, como comunicação em rede, concorrência, sincronização básica e uso de diferentes protocolos de transporte de acordo com a necessidade, resultando em uma solução funcional e alinhada com os objetivos da atividade proposta.

Prints das saídas:

VotingServer



VotingClient

```
Run    VotingServer ×    VotingClient ×

C:\Users\franc\.jdks\openjdk-25\bin\java.exe "-javaagent:D:\IntelliJ IDEA Commu
Digite seu nome:
Werley
Bem-vindo Werley
Candidatos:
- Alice
- Bob
- Carlos
Digite seu voto:
Carlos
Voto registrado!

Process finished with exit code 0
```

segunda-feira, 13 de abril de 2026
seg 00:56 (Hora local)
00:56
13/04/2026

MulticastListener

```
Run    MulticastListener ×    AdminMulticast ×

C:\Users\franc\.jdks\openjdk-25\bin\java.exe "-javaagent:D:\IntelliJ IDEA Commu
NOTA: A votação encerra em brev
```

segunda-feira, 13 de abril de 2026
seg 00:58 (Hora local)
00:58
13/04/2026

AdminMulticast
(apenas envia a mensagem, não tem log)